

OOP | Grundprinzipien

Prinzip	Beschreibung	Java-spezifische Konzepte/Techniken
Abstraktion (Abstraction)	Abstraktion bedeutet, dass nur die relevanten Details eines Objekts sichtbar gemacht werden und irrelevante Details verborgen bleiben.	Abstrakte Klassen mit <code>abstract</code> -Schlüsselwort Schnittstellen (Interfaces) mit dem <code>interface</code> -Schlüsselwort Verbergen von Details durch abstrakte Methoden und Interfaces
Kapselung (Encapsulation)	Das Prinzip, dass die Implementierungsdetails eines Objekts verborgen werden, um eine saubere Trennung zwischen Interface und Implementierung zu gewährleisten.	Verwendung von <code>private</code> und <code>protected</code> -Zugriffsmodifizierern Getter und Setter-Methoden zur Steuerung des Zugriffs auf Felder
Vererbung (Inheritance)	Erlaubt es einer Klasse, von einer anderen zu erben, wodurch sie Eigenschaften und Methoden der übergeordneten Klasse übernehmen kann.	Verwendung von <code>extends</code> , um eine Klasse zu erweitern Polymorphismus durch Überschreibung (<code>@Override</code>) Superklassenmethoden mit <code>super</code> aufrufen
Polymorphismus (Polymorphism)	Polymorphismus erlaubt es Objekten unterschiedlicher Klassen, auf gleiche Weise angesprochen zu werden, obwohl sie unterschiedliche Implementierungen haben.	Überladen von Methoden (<code>method overloading</code>) Überschreiben von Methoden (<code>method overriding</code>) Dynamische Bindung und späte Bindung bei Methodenaufrufen
Konstruktoren (Constructors)	Konstruktoren sind spezielle Methoden, die beim Erstellen eines Objekts aufgerufen werden, um das Objekt zu initialisieren.	Konstruktoren mit dem gleichen Namen wie die Klasse Überladen von Konstruktoren (Multiple Konstruktoren in einer Klasse)
Interfaces	Interfaces definieren Verträge, die von Klassen implementiert werden müssen.	<code>interface</code> -Schlüsselwort für das Erstellen von Interfaces Implementierung von Interfaces in Klassen mit <code>implements</code> Mehrfachvererbung durch Interfaces

Prinzip	Beschreibung	Java-spezifische Konzepte/Techniken
Abstrakte Klassen	Abstrakte Klassen sind Klassen, die nicht direkt instanziiert werden können und oft als Basis für andere Klassen dienen.	Verwendung des <code>abstract</code> -Schlüsselworts für abstrakte Klassen Abstrakte Methoden ohne Implementierung Instanziierung nur über abgeleitete Klassen
Methodenüberladung (Method Overloading)	Methoden mit dem gleichen Namen, aber unterschiedlichen Parameterlisten innerhalb einer Klasse.	Unterschiedliche Parameteranzahl oder -typen bei Methoden mit gleichem Namen
Methodenüberschreibung (Method Overriding)	Das Überschreiben von Methoden aus der Basisklasse, um eine spezifische Implementierung in der abgeleiteten Klasse bereitzustellen.	Verwendung des <code>@Override</code> -Annotations Anpassen der Methode in einer Subklasse, um das Verhalten zu ändern
Komposition (Composition)	Eine Klasse enthält Instanzen anderer Klassen als Mitglieder (besteht aus Objekten anderer Klassen).	Instanziierung von Objekten innerhalb einer Klasse und deren Verwendung als Felder
Assoziationen (Associations)	Beschreibt die Beziehungen zwischen Klassen, etwa „hat-ein“ oder „ist-ein“ Beziehung.	Eine Klasse kann Objekte anderer Klassen referenzieren. Verwendung von Aggregation und Komposition als stärkere Assoziationen
Verborgene Implementierung (Hiding Implementation)	Trennung von API und Implementierung, sodass Änderungen an der Implementierung die Benutzer des Objekts nicht beeinflussen.	Internes Verstecken von Feldern und Methoden durch <code>private</code> Bereitstellung öffentlicher Methoden (API), die auf interne Logik zugreifen
Static- und Instanzmethoden	Unterscheidung zwischen Methoden, die auf Instanzobjekte angewendet werden und Methoden, die auf die Klasse selbst angewendet werden.	<code>static</code> für Methoden, die ohne Instanziierung der Klasse aufgerufen werden können Instanzmethoden, die spezifisch für Objekte sind
Generics (Generische Typen)	Generics ermöglichen es, Klassen, Interfaces und Methoden mit Platzhaltern für Datentypen zu definieren.	Verwendung von <code><></code> -Platzhaltern, um Typen zur Laufzeit festzulegen (z.B. <code>List<T></code>)

Prinzip	Beschreibung	Java-spezifische Konzepte/Techniken
Enums	Enums definieren eine feste Menge von konstanten Werten.	Verwendung des <code>enum</code> -Schlüsselworts, um eine Aufzählung von Konstanten zu erstellen
Innere Klassen (Inner Classes)	Eine Klasse kann innerhalb einer anderen Klasse definiert werden, um den Zugriff auf die äußere Klasse zu erleichtern.	Statische und nicht-statische innere Klassen (mit <code>static</code> für statische innere Klassen) Anonyme und lokale innere Klassen